SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

JadualIQ is a web based AI decision copilot built for busy Malaysians, including part time and gig workers. Rather than simply displaying calendar events, JadualIQ actively analyses a user's weekly schedule alongside real time weather forecasts and travel conditions to identify the single best timing adjustments that saves time and money. By accounting for each user's preferred mode of transport whether ride hailing platform, the system delivers personalized, cost conscious recommendations tailored to their financial situation. JadualIQ transforms a passive calendar into a proactive decision making tool, eliminating the cognitive burden of manual planning and turning scheduling flexibility into a tangible financial advantage.

1.1. Purpose:

This System Analysis Documentation explains the technical structure and design decisions behind JadualIQ. It focuses on:

1. **Architecture**:
   How the frontend, backend modules, GLM layer, external APIs, and data storage are organized.
2. **Data Flows:**
   How schedule input moves through parsing, reasoning, validation, recommendation generation, and user approval.
3. **Model Process:**
   How Z.AI GLM functions as a true service layer for interpretation, reasoning, and explanation.
4. **Role of Reference:**
   This document acts as the technical reference for development, QA, and consistency with the PRD, prototype, and pitch deck.

1.2. Background:

Busy Malaysians often navigate fragmented weekly schedules balanced fixed commitments like classes or work shifts alongside flexible tasks and personal errands. While these schedules may appear manageable, poor timing alignment with real world conditions such as weather changes and ride hailing demand spikes leads to avoidable transport costs and lost time. Manually evaluating whether shifting a flexible task by an hour or two is actually worth it is cognitively demanding and rarely done.

JadualIQ addresses this with a focused, realistic scope: a worker side AI scheduling copilot that identifies the single best same day timing adjustments to minimise transport costs. Rather than attempting to solve all scheduling or transport challenges, JadualIQ does one thing well, analysing a user's existing schedule against estimated Grab fares and real time conditions to surface a smarter, cheaper time to travel. Public transport is included as a reference comparison in the MVP. The goal is to turn scheduling flexibility into a measurable financial and time saving advantage, without adding complexity to the user's day.

1.3. Target Stakeholder

© UMHackathon 2026

Stakeholders	Roles	Expectations
End User	Inputs weekly schedule, reviews recommendation, accepts or rejects change	A practical, trustworthy recommendation that reduces transport cost
Development Team	Builds and integrates the system modules	Clear architecture, modular logic, manageable implementation boundaries
QA Team	Validates behavior, fallbacks, and recommendation accuracy	Defined flows, predictable output structure, testable edge cases
Judges / Reviewers	Evaluate feasibility, AI integration, and technical clarity	Evidence that GLM is central and architecture is well reasoned

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
Architecture	Client server architecture with multiple functional modules inside one backend application
Core Reasoning Layer	Z.AI GLM service integration
Validation Layer	Deterministic rules and output verification
External APIs	Weather API, Google Maps API / traffic informed estimation
Storage	Logical relational data model using SQLite

JadualIQ is built as a web based client server system. The user interacts through a chatbot and calendar interface on the frontend. The backend is structured as multiple internal modules rather than independent distributed microservices. This keeps the architecture realistic for a hackathon MVP while still allowing clear separation of responsibilities.

Main Backend Modules

1. **API Controller Module**
   Receives frontend requests and returns structured responses through routes such as /api/chat, /api/schedule, /api/conflicts/resolve, /api/restrictions, and /api/impact.
2. **Schedule Processing Module**
   Parses and normalizes natural language schedule input into structured fields such as task name, date, time, duration, location, origin, destination, and priority score. In the documentation, this is handled mainly through the Intake Agent powered by Z.AI GLM, with regex fallback if the API fails.
3. **Recommendation Engine Module**
   Combines parsed schedule data, weather context, traffic context, calendar conflicts, user restrictions, and priority score, then asks GLM to produce the final scheduling recommendation.
4. **GLM Orchestration Module**
   Constructs prompts, calls the Z.AI GLM API, retries failed calls, repairs/parses JSON output, and returns structured AI responses to the backend.
5. **Validator Module**
   Checks whether inputs and outputs are scheduling related, whether required fields are missing,

© UMHackathon 2026

whether returned JSON is usable, and whether the system needs to fall back to clarification, warning, or mock response behavior.

6. **External Data Integration Module**
   Retrieves transport and environmental context using OpenWeatherMap and Google Maps Distance Matrix API. The documentation also notes that the traffic agent uses the actual scheduled departure time epoch instead of "now," which strengthens the technical correctness of the transport recommendation.
7. **Persistence Module**
   Stores users, tasks, restrictions, schedule status, priority, rationale, and estimated RM savings using SQLite through schedule_store.py.

This multi module backend is appropriate for a preliminary round MVP because it shows clear engineering structure without overclaiming cloud native complexity. That also aligns with the judging criteria, where code modularity, system logic, workflow integration, and implementation quality are directly evaluated.

2.1.2 LLM as Service Layer

Z.AI GLM is integrated as a core reasoning service layer, not as a decorative AI chatbot.

GLM Responsibilities

- Interpreting messy natural language schedule input
- Identifying task intent and missing schedule fields
- Assigning contextual priority
- Resolving conflicts between tasks
- Reasoning over weather, traffic, restrictions, and existing calendar context
- Deciding whether the schedule should be confirmed, warning, or blocked
- Recommending transport or timing alternatives
- Generating plain language explanations for the user

Deterministic Layer Responsibilities:

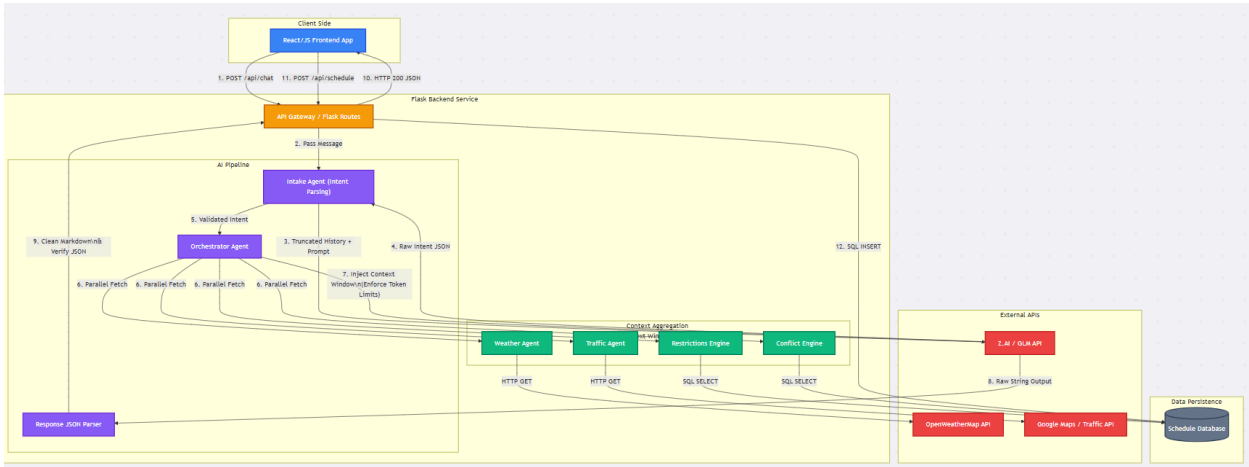
- routing frontend requests
- checking whether the message is scheduling related
- enforcing required field checks before orchestration
- calling external APIs for weather and traffic context
- scanning SQLite for schedule conflicts
- checking user restrictions and blocked time windows
- parsing, repairing, and validating GLM JSON output
- handling retries, fallbacks, and graceful degradation
- storing confirmed tasks, rationale, status, priority, and estimated savings

This separation is critical. If GLM were removed, JadualIQ would lose its meaningful ability to interpret unstructured input, reason through ambiguous scheduling trade offs, and generate understandable decision explanations. The remaining deterministic logic would only

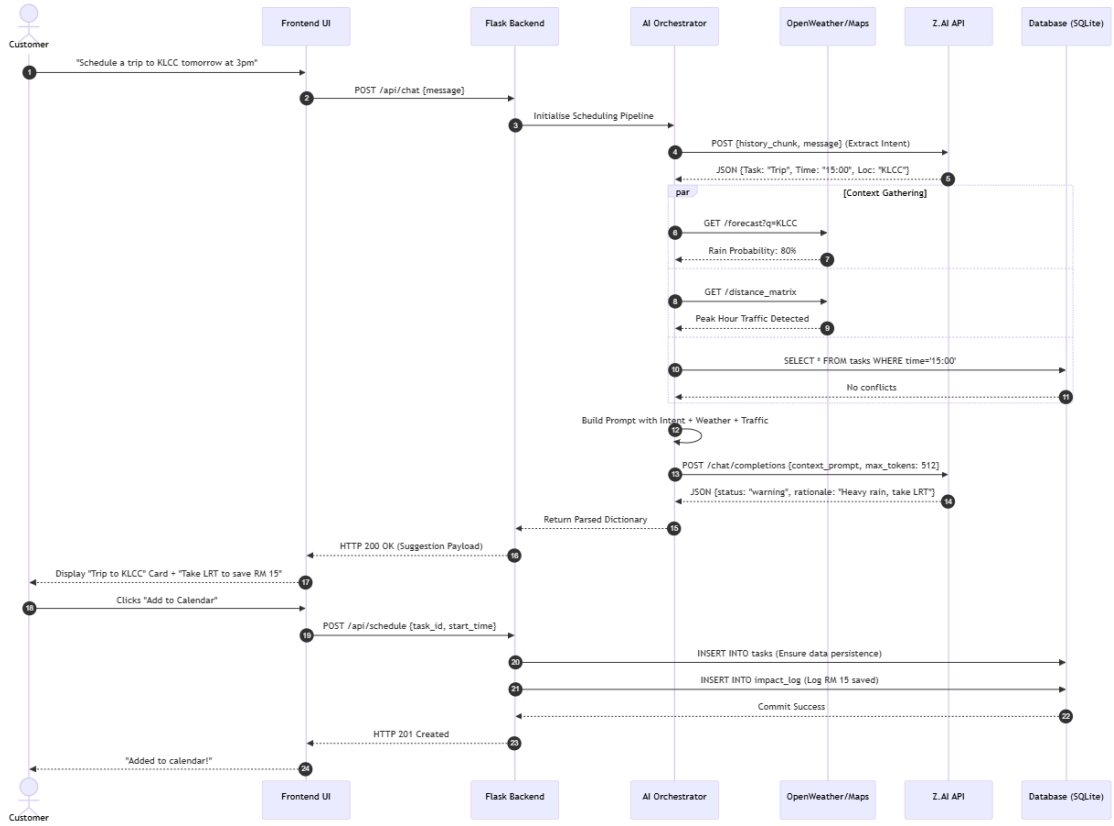
© UMHackathon 2026

provide partial rule execution, not full decision intelligence. That is exactly the distinction the domain brief expects.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

2.2.1. Vanilla JavaScript (HTML5 + CSS3 + ES6 Modules)

JadualIQ uses a lightweight browser native frontend without a heavy framework. Static frontend files are served directly by the Flask backend, keeping deployment simple and avoiding the need for a separate build pipeline. The frontend uses JavaScript modules such as chat.js, calendar.js, and impact.js to handle chat interaction, calendar rendering, and impact statistics updates.

2.2.2. Backend: Flask 3.03

The backend is built using Flask, a lightweight Python web framework suitable for rapid prototyping and AI agent workflow orchestration. It exposes REST style API endpoints under /api/*, with modular route files for authentication, chat, schedule management, conflict resolution, restrictions, and impact tracking. The backend coordinates the main workflow across intake parsing, clarification handling, weather lookup, traffic lookup, conflict checking, constraint checking, GLM orchestration, JSON parsing, and response delivery.

2.2.3. SQLite (file based relational database — jadualiq.db)

JadualIQ uses SQLite as a file based relational database for hackathon scoped persistence. It stores users, schedule tasks, restrictions, task status, priority, rationale, and estimated RM savings. SQLite is suitable for a single server MVP because it is simple to configure and requires no separate database server. For production, the database layer could be migrated to PostgreSQL with proper schema migration and deployment changes.

2.2.4. Local/Self hosted (Flask dev server)

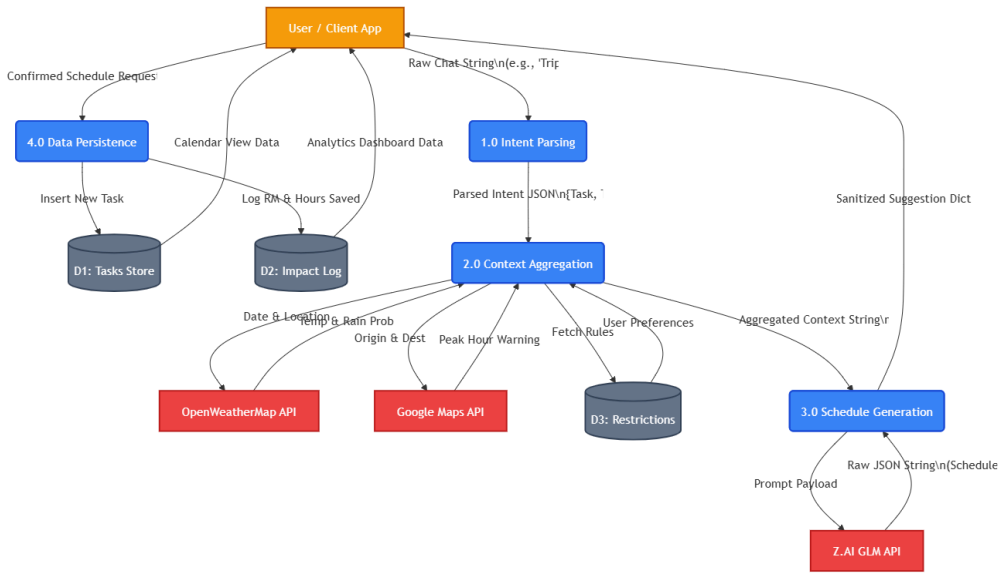
The current MVP is designed for local or self hosted execution using the Flask server. No cloud native deployment is claimed at the preliminary stage. For production, the Flask app could be hosted on a VPS or AWS EC2, SQLite could be replaced with PostgreSQL on RDS, static frontend assets could be served through Nginx or object storage, and environment secrets could be managed through .env files or a dedicated secrets manager

2.3. Key Data Flows

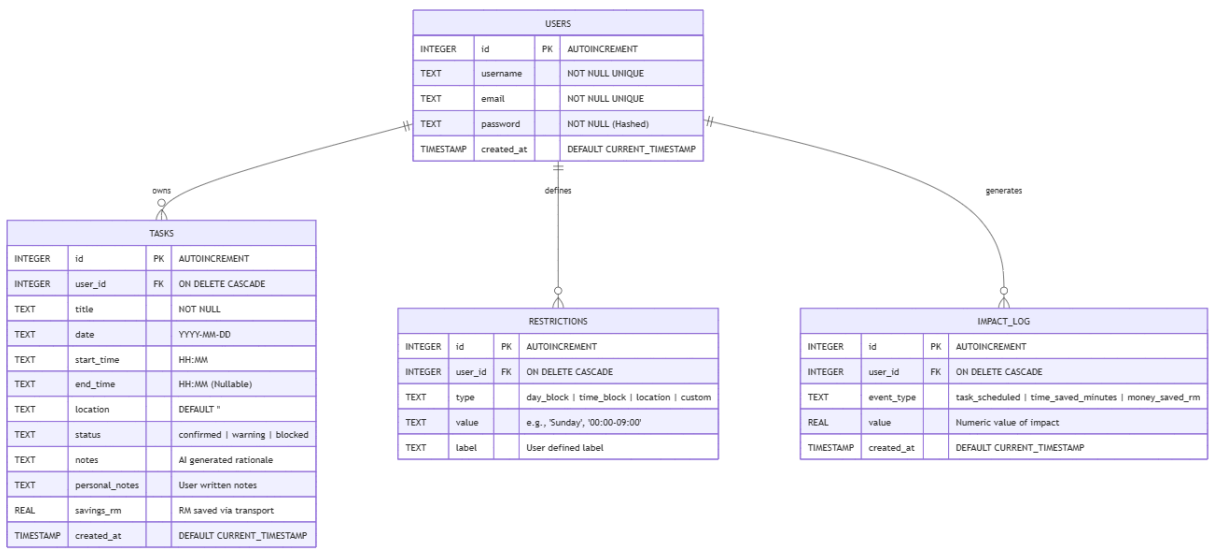
2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

© UMHackathon 2026



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

This section highlights the core boundaries of the project so JadualIQ remains a high impact MVP for the preliminary round and matches the hackathon's judging focus on technical feasibility and product clarity.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description
1	Natural Language Schedule Intake	User enters weekly commitments in chatbot and system structures them
2	Weekly Calendar Structuring	Parsed schedule is shown in a calendar UI
3	Same Day Recommendation Engine	One best cheaper same day switch is generated when feasible
4	Weather Surge Risk Awareness	System detects weather related conditions that may increase likely ride hailing cost
5	Approval Based Rescheduling	User accepts recommended change or keeps original schedule

3.2. Non Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Reliability	The system must handle external API failures from Z.AI GLM, OpenWeatherMap, and Google Maps without crashing. Invalid or inconsistent schedule suggestions should not be shown to the user.	Retry logic is used for GLM calls, fallback responses are provided when APIs fail, and the validator/JSON repair layer checks whether AI output is usable before returning it to the frontend.
Maintainability	Backend logic should be modular so new agents or services can be added without rewriting the entire system. Each major workflow component should be testable in isolation.	The backend is separated into routes, agents, models, and configuration modules, such as chat.py, schedule.py, orchestrator.py, weather.py, traffic.py, and schedule_store.py.

© UMHackathon 2026

Explainability	Users must understand why a switch is recommended. New agents must include a human readable rationale with weather, traffic and RM savings.	GLM returns a plain language explanation, rationale, schedule status, transport context, and estimated RM savings, which can be displayed in the chat bubble or explanation panel.
Fault Tolerance	Partial failure of weather, traffic, or GLM services should not fully break the scheduling flow. The system should continue with limited output when possible.	Weather and traffic agents can return simulated or fallback data, while GLM failures can trigger mock or graceful fallback responses. The user is informed when the recommendation is based on limited data.
Cost Efficiency	GLM usage should stay bounded by limiting unnecessary prompts, repeated calls, and oversized context. Off topic messages should not trigger the full AI scheduling pipeline.	The Intake Agent uses only the last 6 chat messages to reduce context size, guardrails filter non scheduling queries, and the system keeps prompt tasks focused instead of sending the full user history.
Latency	The chat endpoint should return a usable response within an acceptable time window under normal MVP load. Slow AI or external API responses should not leave the user without feedback.	GLM calls use retry limits and timeout settings. The frontend can show loading states while the backend completes the agent pipeline.
Action Safety	The system must not automatically change the user's schedule without user approval. Recommendations should remain suggestions until the user confirms them.	Schedule changes are gated through frontend approval and persisted only after the user accepts or creates the task.

3.3. Out of Scope / Future Enhancements

© UMHackathon 2026

1. Multi day rescheduling
2. More alternative rescheduling options if the user rejects one
3. Full public transport optimization
4. Automatic shift acceptance
5. Multi user collaboration
6. Long term analytics
7. Mobile native app
8. Gig platform integration
9. Employer side scheduler

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

5. Grayscale Rollout & A/B Testing:

Since JadualIQ is a hackathon stage MVP and not a live production deployment, the rollout strategy is described as a controlled staged release simulation rather than an actual public traffic rollout. The purpose of this approach is to reduce risk while validating whether the recommendation logic, explanation quality, and fallback behavior remain stable under gradual exposure.

The proposed grayscale rollout strategy is:

- Stage 1 – Internal Developer Testing
  - The feature is first tested only by the development team using predefined weekly schedule scenarios.
  - At this stage, focus is placed on schedule parsing, same day rule enforcement, validator behavior, and recommendation card generation.
  - Any issue involving broken timing references, invalid schedule changes, or incorrect recommendation labels blocks progression.
- Stage 2 – Controlled Scenario Testing
  - The feature is then tested against the 3 locked demo scenarios:
    - Normal weekly planning case
    - Weather triggered cheaper same day recommendation
    - Missing information / conflict / limited data case
  - This stage verifies that the recommendation engine produces stable outputs against the baseline of the original schedule unchanged.
- Stage 3 – Limited Demo Exposure
  - The system is exposed only to a small set of demo users or evaluators in a monitored demo environment.
  - At this stage, JadualIQ is evaluated for usability, recommendation clarity, and whether fallback labels such as Limited Recommendation are shown correctly when external data is incomplete.
- Stage 4 – Expanded Demo Readiness

© UMHackathon 2026

- If no critical defects are detected, the same feature flow is used for wider judging/demo presentation.
- This does not imply production scaling, but rather readiness for more repeated demonstration under controlled conditions.

For A/B testing, JadualIQ does not treat this as a product marketing experiment. Instead, A/B testing is used in a functional evaluation sense to compare two recommendation presentation or reasoning strategies in a controlled environment.

Proposed A/B test candidates:

- Variant A: recommendation explanation emphasizes estimated savings first
- Variant B: recommendation explanation emphasizes weather/transport risk first

5.1.

5.1.1. **Emergency Rollback (ER) & Golden Release**:

Because JadualIQ is a decision support system, rather than an autonomous scheduler, our emergency behavior must prioritize safety, consistency, and preservation of the original schedule over forcing the system to act on uncertain outputs. The user always remains the final decision maker.

The emergency rollback strategy is:

- Forecast accuracy safeguard: When a user schedules an event more than 7 days in the future, the system does not present its weather and traffic guidance as authoritative. Instead, a Forecast Accuracy Notice is displayed on the event card, advertising the user to review the recommendation closer to the date.
- Restriction-aware generation: User-defined restrictions (blocked days, blocked time ranges and custom plain-text rules → this is managed by the My Restrictions panel) are passed to GLM as part of the prompt, so the model is instructed to respect them during recommendation generation rather than relying on later filtering.
- Mandatory user approval: No schedule modification is ever applied automatically. Every recommendation appears as a suggestion card with explicit Add to Calendar and Dismiss buttons and only "Add to Calendar" promotes the event to the Confirmed status. If the user dismisses or ignores a suggestion, the original schedule is left untouched.
- Input validation before GLM: User inputs are validated before being sent to GLM to prevent malformed or empty prompts from reaching the model. (Post-generation validation of output is not implemented yet in the current MVP and is considered to be implemented in the planned hardening step).

For JadualIQ, the Golden Release should be defined as the last stable version of the product flow that can still:

- Parse schedule input and restrictions correctly and successfully

© UMHackathon 2026

- Display the weekly calendar correctly
- Preserve the original schedule safely
- Show either a valid recommendation or a safe fallback state
- Prevent invalid changes from being applied

This means the golden version is not necessarily the most feature rich version, but the most reliable demo safe version.

Recommended emergency triggers:

- Invalid time or schedule references in the recommendation
- GLM returns malformed JSON or response that cannot be parsed by the backend
- Mismatch between explanation and estimated cost fields
- Weather/maps dependency failure causing unusable output
- Approval flow applying a change incorrectly

Recommended rollback actions:

- Stop the current recommendation from being displayed, if the recommendation card is inconsistent or considered broken (empty or inconsistent).
- Revert to the last validated recommendation state if available
- Otherwise fall back to original or Limited Recommendation
- Preserve the original weekly schedule as the source of truth, meaning never silently modify confirmed events.

5.1.2. Priority Matrix:

5.1.2.1. We classify the incidents into four priority levels based on impact on user trust and schedule integrity. Each level pairs a measurable trigger with a specific action and response window.

Priority	Trigger Condition	Response Action	Response Time
1 - Critical	When Vercel deployment is fully unreachable or when GLM returns malformed/unparseable JSON on more than 5% of requests over 5 minutes or when a confirmed event is silently deleted or modified without user approval.	Trigger instant rollback to last Golden Release via Vercel's rollback feature and disable suggestion generation until the incident is resolved, notify the team so it can work ASAP.	Immediate (< 15 min)
2 - High	OpenWeather or GoogleMapsAPI unavailable, causing recommendations	Suppress the affected	Within 2 hours

© UMHackathon 2026

Priority	Trigger Condition	Response Action	Response Time
	to generate without user weather/transport context or GLM ignores user restrictions (suggests an event on a blacklisted day)	suggestion card rather than display information advice and also display 'context unavailable' notice in the AI advice panel inside the schedule	
3 - Medium	Forecast accuracy notice fails to appear on events more than 7 days or weekly impact sidebar (Hours Saved/RM saved / Active conflicts) does not update after Add to Calender	Log issue, add to current sprint, communicate workaround to affected users if any	Within 1 week
4 - Low	Cosmetic UI issues (alignment, icon spacing), typos in AI generated explanations	Add to issue tracker , fix in next regular release cycle	Next release

5.2. **Monitoring**:

5.2.1. Planned monitoring categories:

- GLM response validity rate
- external API success/failure rate
- validator rejection rate
- recommendation generation success
- approval action completion
- frontend display failures

5.3. **Assumptions:**

a. Users can provide understandable natural language schedule input
b. Target day schedule has enough timing detail for evaluation
c. Weather and Google Maps context are sufficiently available for MVP demonstration
d. One active week is enough for preliminary round use
e. The original schedule is the comparison baseline

5.4. **External Dependencies:**

© UMHackathon 2026

Tools	Purpose	Risks
Z Ai GLM API	Serves as JadualIQ's core reasoning engine for interpreting messy schedule input, comparing same day timing options, and generating user friendly recommendation explanations. It is the main component that turns raw schedule and external context into decision support output rather than simple rule execution.	High - if the GLM API is unavailable, rate limited, or returns unusable output, JadualIQ loses its main reasoning capability and becomes much weaker as a decision intelligence product. This risk is especially critical because the product depends on GLM for interpretation, trade off reasoning, and explanation, not just chatbot interaction.
OpenWeather API	Provides weather condition signals, especially rain related context, that help JadualIQ identify when the user's original timing may face higher transport cost risk. This allows the system to support weather triggered recommendations instead of relying only on static schedule comparison.	Medium - if weather data is missing, delayed, or inaccurate, JadualIQ may miss a weather related surge risk situation or reduce the confidence of its recommendation. However, the product can still continue with a limited recommendation because weather is an important supporting factor, not the only basis of the decision.
Google Maps API	Supplies travel and traffic informed context used to support the API informed estimated ride hailing cost comparison between the original timing and the suggested same day timing. This helps JadualIQ ground its recommendation in route and timing conditions rather than making purely abstract schedule judgments.	High - if Google Maps or related traffic informed estimation is unavailable or incomplete, the system's cost comparison becomes weaker and less defensible, which directly affects the recommendation quality. Since transport savings is the product's main measurable value, weak travel estimation creates a major credibility risk for the MVP.

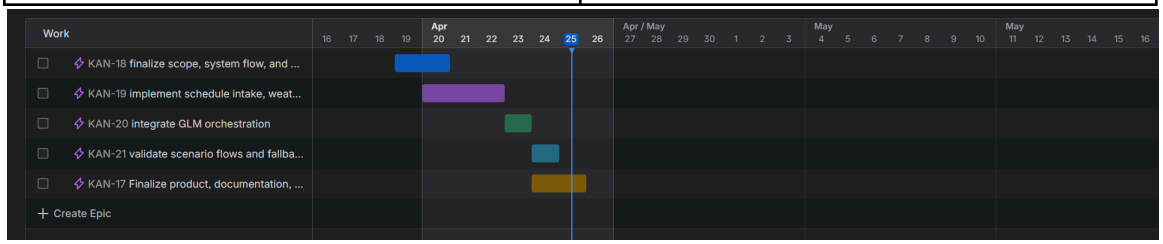
6. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two week sprint.

© UMHackathon 2026

6.1. Project Timeline:

Day 1 - 2	finalize scope, system flow, and document direction
Day 2 - 4	implement schedule intake, weather/ maps context, UI UX, database, and start documentation
Day 5	integrate GLM orchestration
Day 6	validate scenario flows and fallback handling
Day 6 - 7	Finalize product, documentation, pitch flow, and demo polish



6.2. Team Members Role:

- Frontend lead: Matthew Wijaya
- Backend lead: Mario Oey
- AI / integration lead: Darren Melvern
- QA / documentation lead: Dave Alexander Can
- Product / pitch lead: Dave Alexander Can

6.3. Recommendations:

- Keep the backend modular but not over distributed
- Implementation of mobile app
- Future Analytic
- Prioritize validator accuracy over adding more features
- Retain snapshot logging of recommendation inputs for explainability
- Polish one best recommendation flow before expanding scope

References:

© UMHackathon 2026

Email: umhackathon@um.edu.my

Thorhauge, M., Swait, J., & Cherchi, E. (2020). The habit-driven life: Accounting for inertia in departure time choices for commuting trips. *Transportation Research Part a Policy and Practice*, *133*, 272–289. https://doi.org/10.1016/j.tra.2020.01.023

Wong, A. (2022, May 27). Why is eHailing so expensive? Here's what Grab and AirAsia Ride have to say about surge pricing. SoyaCincau. https://soyacincau.com/2022/05/27/why-is-ehailing-malaysia-so-expensive-these-days-grab-and-airasia-ride-responds/

© UMHackathon 2026

Email: umhackathon@um.edu.my